A Short Tour of Transformer Expressivity From Hard Attention to Soft Attention

Andy Yang (Univ. of Notre Dame, USA)

30 July 2024

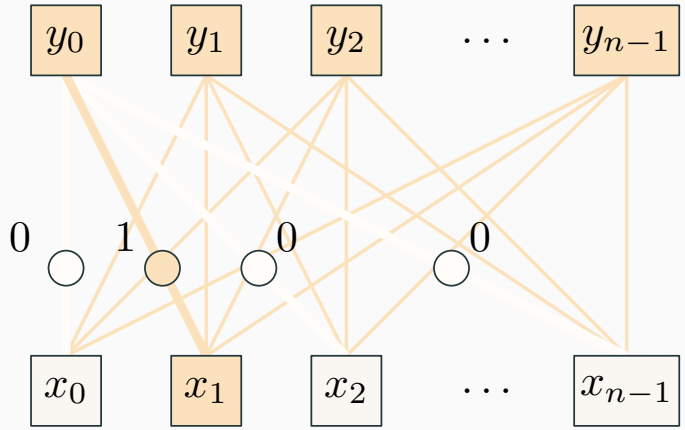
Today's goals and question

Today's Goals

1. **Explain** the difference between masked hard-attention transformers and soft attention transformers
2. **Describe** how strict-masking, positional embeddings, and depth contribute to the expressivity of a masked hard-attention transformer
3. **Demonstrate** how the same idea can be applied to softmax transformers
4. **Discuss** the issues with these bounds on expressivity

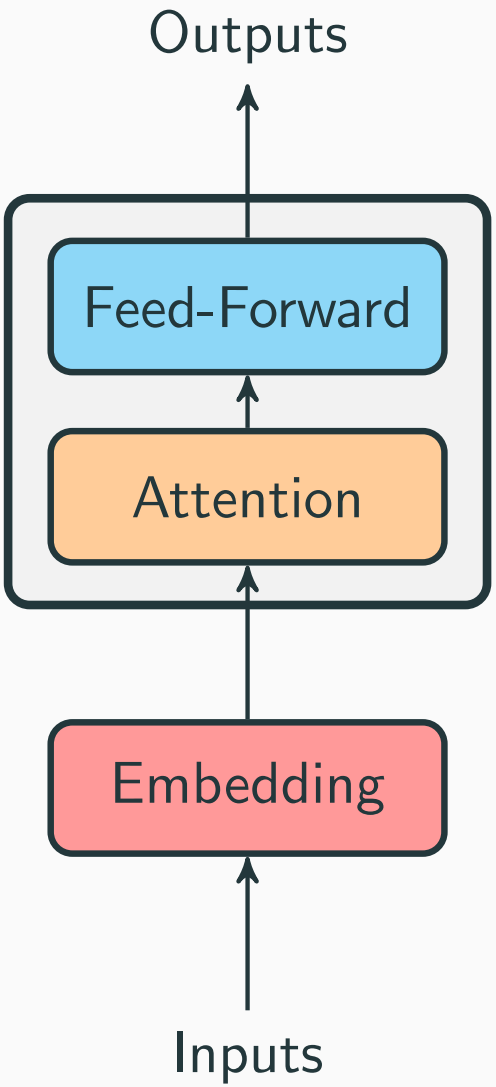
Main Idea

Unique-hard attention transformers correspond closely with LTL

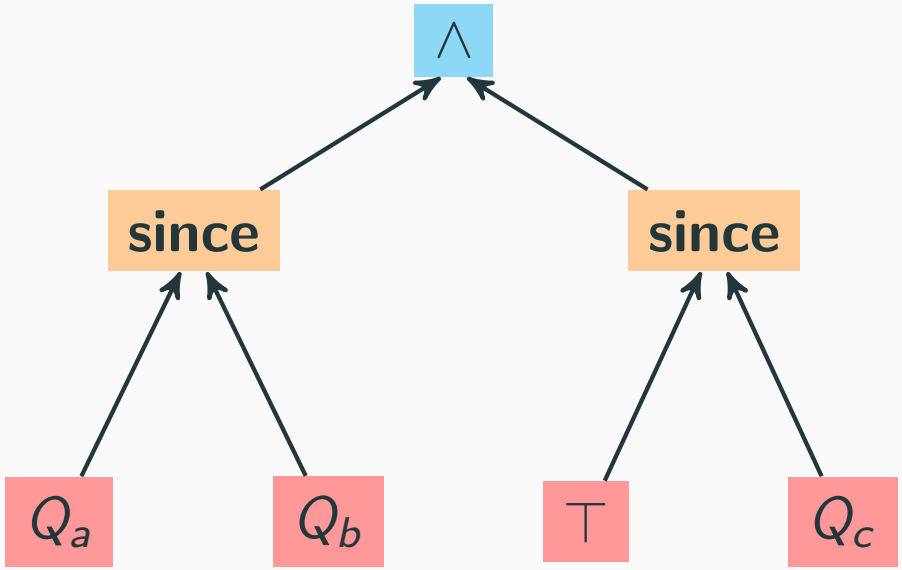


This allows us to bring results from one domain to the other.

LTL **Correspondence**



$w \vDash (Q_a \textbf{ since } Q_b) \wedge (\top \textbf{ since } Q_c)$



$w \in \Sigma^*$

Analyzing masked hard-attention transformers

Logic and Computation

Logic is a powerful tool for describing computation.

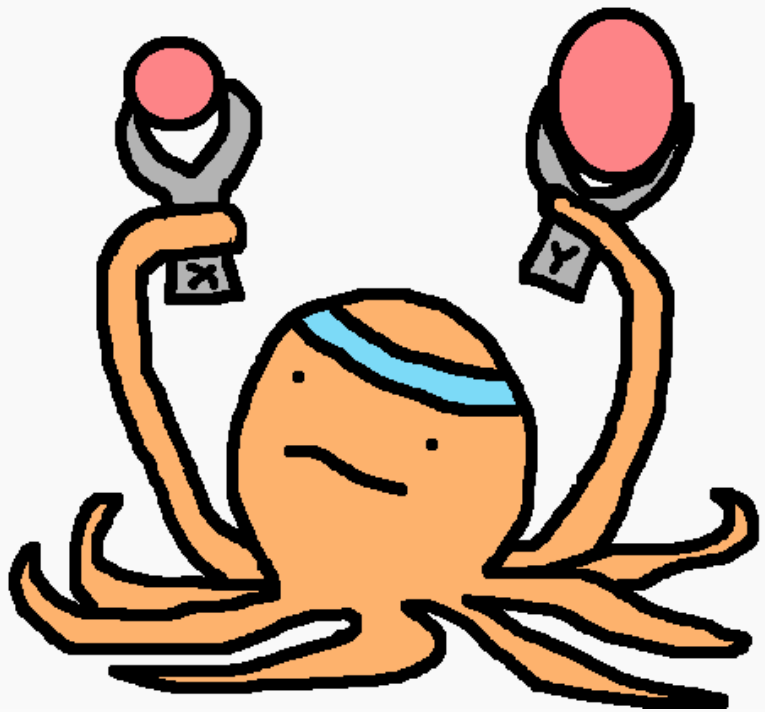
- Quantifier depth corresponds to parallel computation time
- Number of variables corresponds to number of processors

Quantification and Parallelism

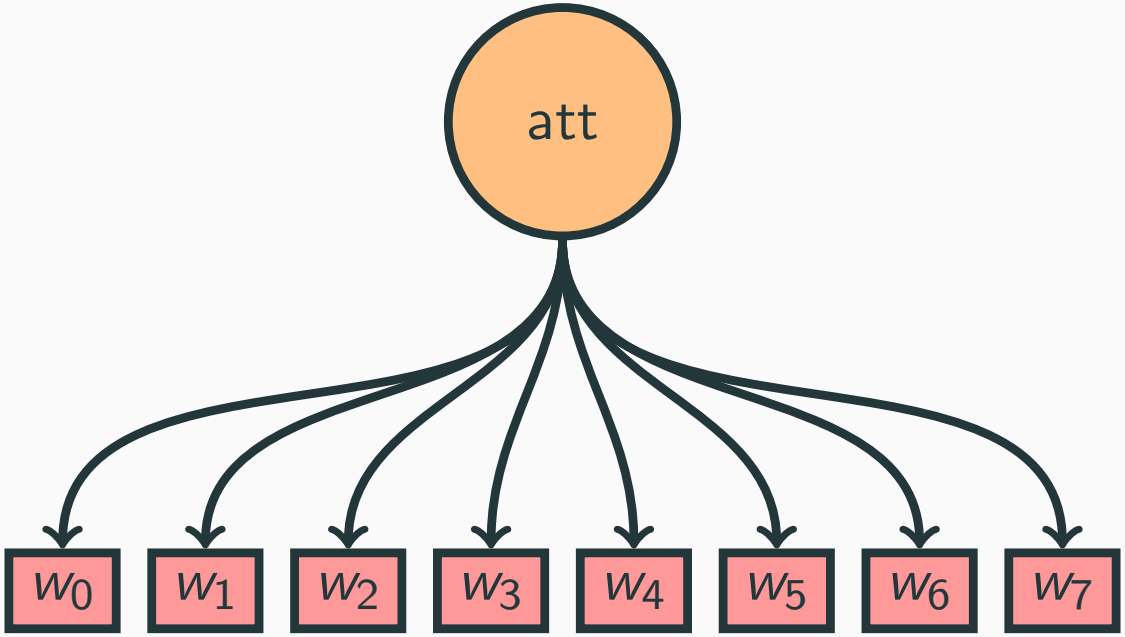
$$\text{otto} \models \forall x.\text{ball}(x)$$

Number of Variables and Number of Processors

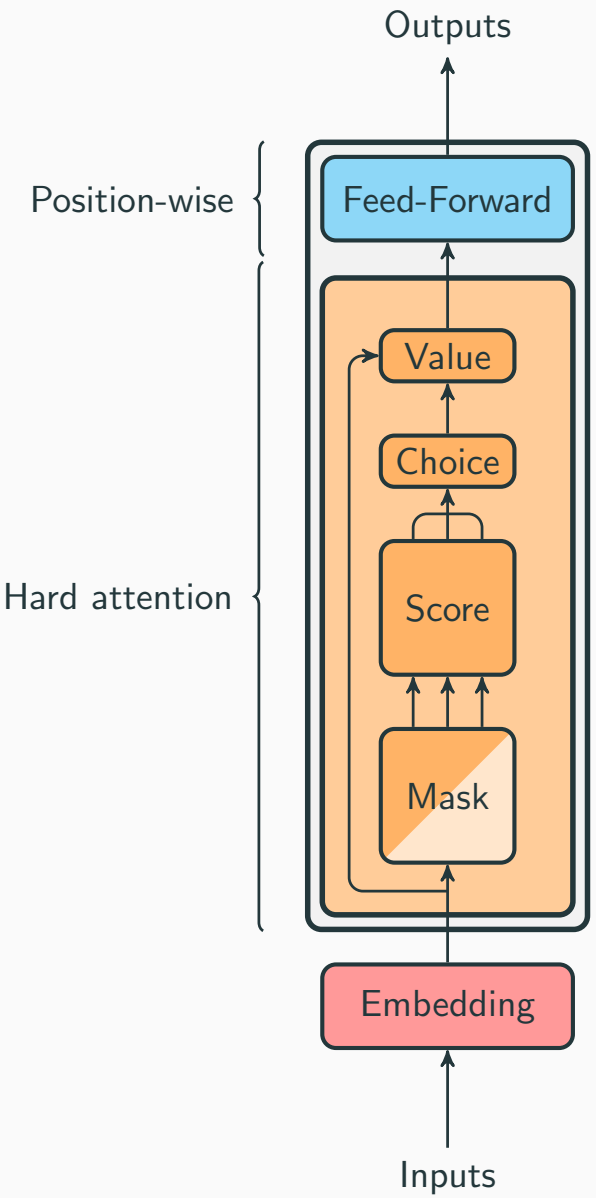
$$\text{otto} \models \text{size}(x) < \text{size}(y)$$



Self-Attention



Masked Hard-Attention Transformers



3-layer feed-forward network with ReLU activation

Values: any linear function with residual connection

Leftmost or rightmost maximum

Attention scores: any bilinear function

Attention masking: strict future, strict past, or none

Rightmost Hardmax

Definition (Hardmax)

The n-dimensional rightmost hardmax function is the function $\text{rhardmax}\colon \mathbb{R}^n \to \mathbb{R}^n$ such that

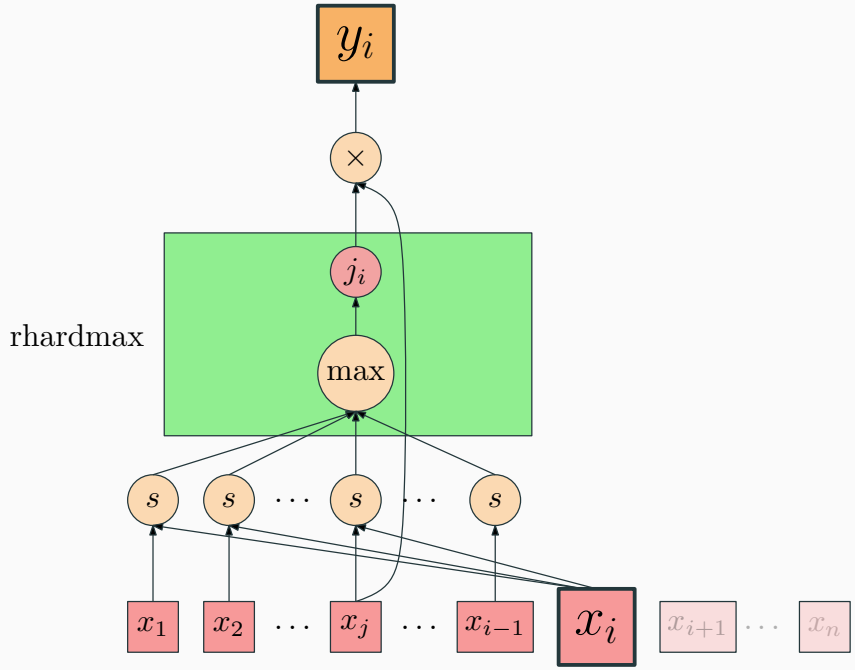
$$[\text{rhardmax}(s)]_i = \mathbb{I}[i = \max I]$$
$$I = \{i \in [n] \mid s_i = \max s\}.$$

That is, it returns the one-hot vector with a 1 in the *rightmost* coordinate that maximizes s.

Masked Unique Hard Self-Attention (in case needed)

Masked unique hard self-attention with d input/output dimensions and d_{hid} key/value dimensions is a function

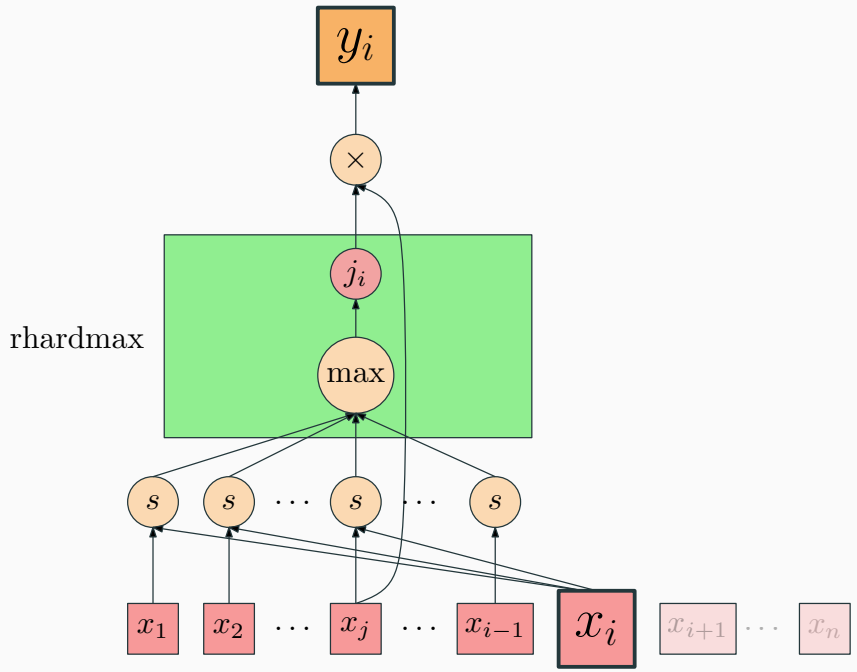
$$
\begin{aligned}
\mathit{att}\colon (\mathbb{R}^d)^* &\xrightarrow{\text{lp}} (\mathbb{R}^d)^* \\
\mathbf{X} &\mapsto \mathbf{Y} \\
\mathbf{Q}[i] &= \mathbf{W}^{\mathsf{Q}}(\mathbf{X}[i]) \\
\mathbf{K}[j] &= \mathbf{W}^{\mathsf{K}}(\mathbf{X}[j]) \\
\mathbf{V}[j] &= \mathbf{W}^{\mathsf{V}}(\mathbf{X}[j]) \\
\mathbf{s}[i,j] &= \frac{\mathbf{Q}[i] \cdot \mathbf{K}[j]}{\sqrt{d_{\text{hid}}}} \\
\alpha[i,:] &= \text{rhardmax}(\mathbf{s}[i,:]) \\
\mathbf{Y}[i] &= \sum_{j \in [n]} \alpha[i,j]\, \mathbf{V}[j]
\end{aligned}
$$



Masked Unique-Hard Attention Distilled

For each $\mathbf{X}$, do the following to get $att(\mathbf{X})$:

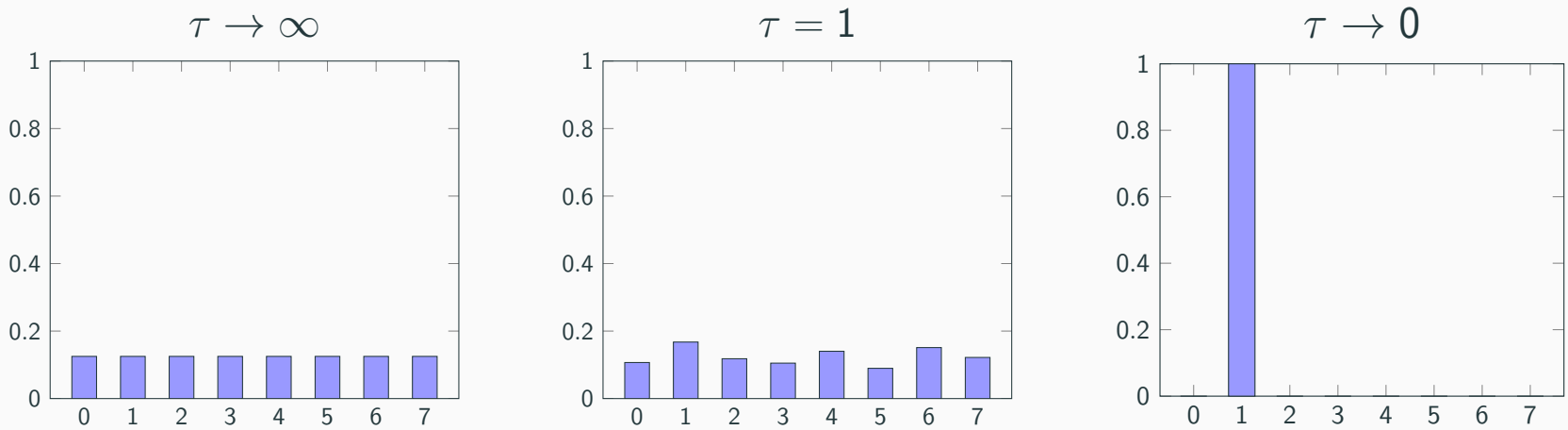
1. If $i = 0$, return $\mathbf{0}$
2. Compute $\mathbf{s}[i, j] = \frac{\mathbf{Q}[i] \cdot \mathbf{K}[j]}{\sqrt{d_{\text{hid}}}}$ for each $j < i$
3. Let $j_i = \text{rhardmax}_j \, \mathbf{s}[i, j]$
4. Return $\mathbf{V}[j_i]$



Hard Attention

At low temperature, softmax closely approximates rhardmax – selecting a single input (if there is a unique maximum or tiebreaking is enforced).

$$\text{softmax}(x_i, \tau) = \frac{e^{x_i/\tau}}{\sum_{j=0}^{n} e^{x_j/\tau}}$$



Hard Attention in the Wild

- Real transformers are often observed to focus attention on a very small number of positions [Merrill et al., 2021].
- On Dyck languages, they have been found to learn effectively unique-hard attention in their second layer [Ebrahimi et al., 2020, Fig. 1]. [Chiang and Cholak, 2022], but in practice, transformers cannot learn parity [Bhattamishra et al., 2020]. Unique-hard attention transformers also cannot compute parity [Hahn, 2020], so they are in some sense more realistic.
- Hard attention has occasionally been used in practice in previous research on interpretability [Kinley, 2020] and efficiency [Gupta et al., 2021, Xu et al., 2021].

Question of the Day

What is the right sort of quantification and amount of variables to describe (unique-hard) transformer computation?

Equivalence

Main Result

B-RASP

B-RASP: example program

$$
\begin{array}{rcl}
Q_\ell & & \\
Q_r & & \\
P_\ell(i) & := & \blacktriangleright_j\,[j < i, 1]\;\; Q_\ell(j) : 0 \\
S_r(i) & := & \blacktriangleleft_j\,[j > i, 1]\;\; Q_r(j) : 0 \\
I(i) & := & (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)) \\
B_\ell(i) & := & \blacktriangleright_j\,[j < i, \neg I(j)]\;\; Q_\ell(j) : 0 \\
A_r(i) & := & \blacktriangleleft_j\,[j > i, \neg I(j)]\;\; Q_r(j) : 0 \\
C(i) & := & I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)) \\
Y(i) & := & \blacktriangleright_j\,[1, \neg C(j)]\;\; 0 : 1
\end{array}
$$

B-RASP: program trace

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ						
Q_r						
P_ℓ						
S_r						
I						
B_ℓ						
A_r						
C						
Y						□

B-RASP: attention operators

$$\blacktriangleright [j < i, S(i,j)] \; V(j) : D(i)$$

find rightmost j left of i such that $S(i,j)$ and return $V(j)$ or $D(i)$ if no such j exists

B-RASP: attention operators

$$\blacktriangleleft [j > i, S(i,j)] \; V(j) : D(i)$$

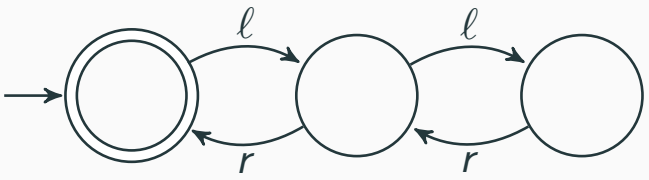
find leftmost j right of i such that $S(i,j)$ and return $V(j)$ or $D(i)$ if no such j exists

Initial Vectors - B-RASP Example: Dyck-1 of Depth 2

The initial vectors are Q_ℓ and Q_r. These are all defined such that:

$Q_\ell(i)$ = True iff ℓ is i-th symbol

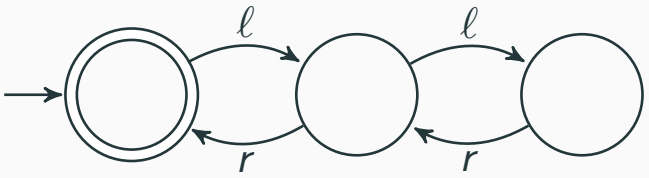
$Q_r(i)$ = True iff r is i-th symbol



Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1

P_ℓ - B-RASP Example: Dyck-1 of Depth 2



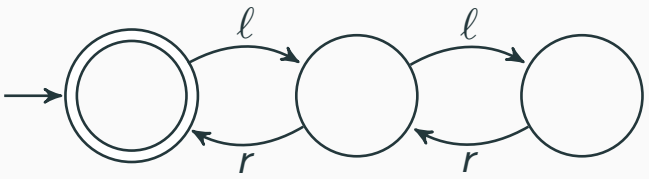
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j\,[j < i, 1]\;\; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ						

$P_\ell : i = 1$



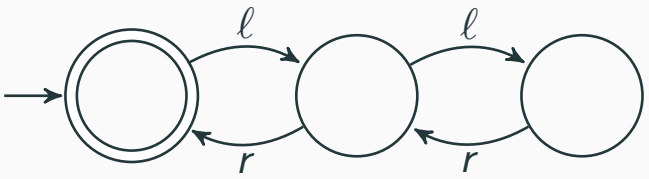
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	? $_i$					
score	–	–	–	–	–	–

$P_\ell : i = 1$



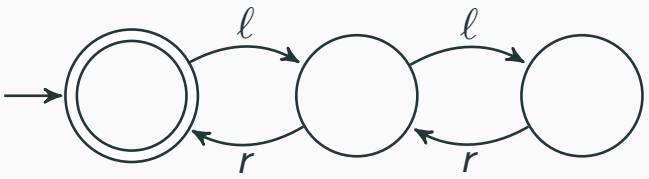
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0 $_i$					
score	–	–	–	–	–	–

$P_\ell : i = 2$



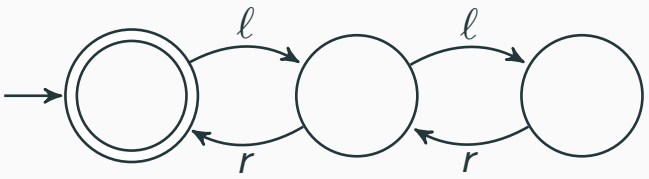
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1_j	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	$?_i$				
score	1_j	–	–	–	–	–

$P_\ell : i = 2$



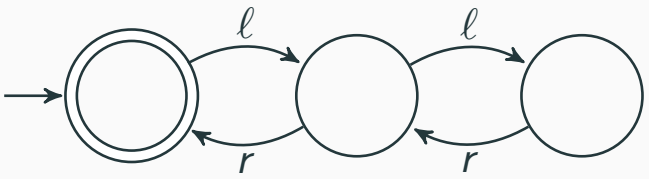
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1_j	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	$?_i$				
score	1_j	–	–	–	–	–

$P_\ell : i = 2$



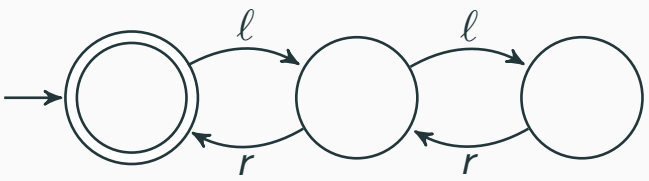
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1_j	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1_i				
score	1_j	–	–	–	–	–

$P_\ell : i = 3$



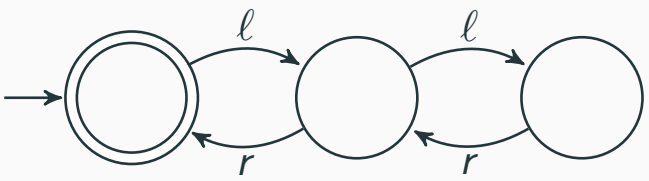
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1_j	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	$?_i$			
score	1	1_j	–	–	–	–

$P_\ell : i = 3$



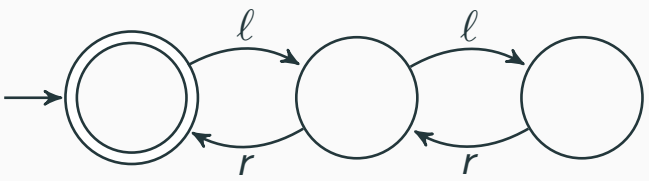
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1_j	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	$?_i$			
score	1	1_j	–	–	–	–

$P_\ell : i = 3$



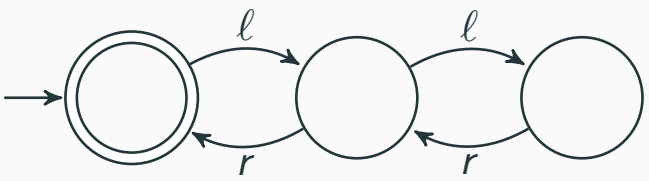
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1_j	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1_i			
score	1	1_j	–	–	–	–

$P_\ell : i = 4$



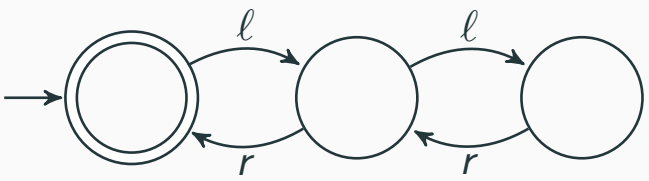
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0_j	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	$?_i$		
score	1	1	1_j	–	–	–

$P_\ell : i = 4$



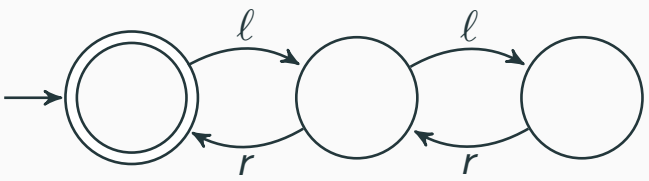
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0_j	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	$?_i$		
score	1	1	1_j	–	–	–

$P_\ell : i = 4$



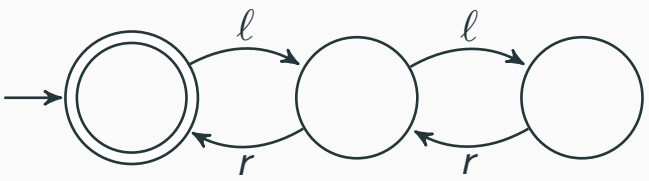
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0_j	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0_i		
score	1	1	1_j	–	–	–

$P_\ell : i = 5$



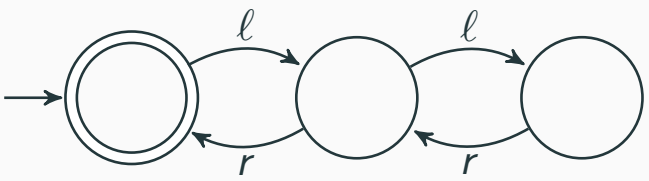
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1_j	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	$?_i$	
score	1	1	1	1_j	–	–

$P_\ell : i = 5$



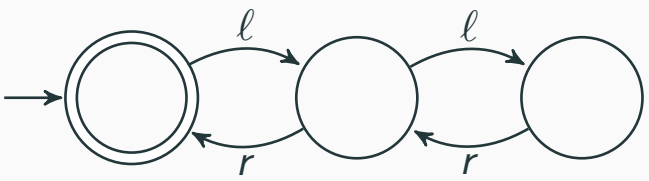
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1_j	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	$?_i$	
score	1	1	1	1_j	–	–

$P_\ell : i = 5$



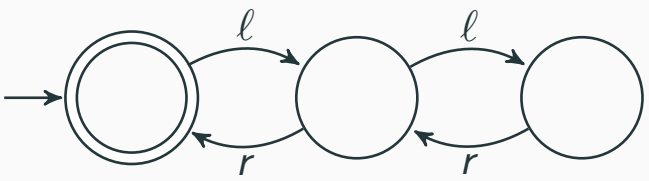
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1_j	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1_i	
score	1	1	1	1_j	–	–

$P_\ell : i = 6$



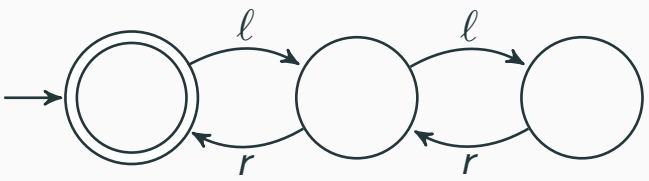
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0_j	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	$?_i$
score	1	1	1	1	1_j	–

$P_\ell : i = 6$



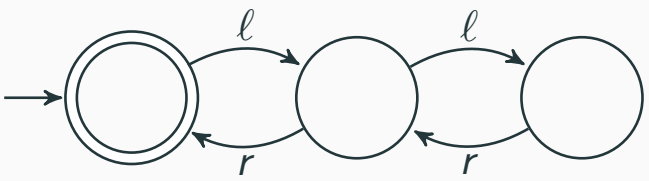
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0_j	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	$?_i$
score	1	1	1	1	1_j	–

$P_\ell : i = 6$



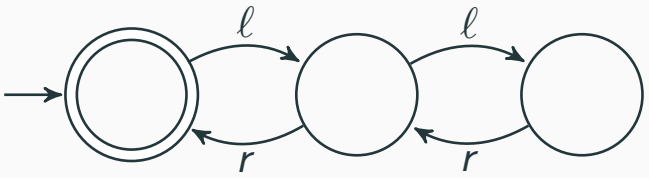
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j [j < i, 1] \; Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0_j	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	0_i
score	1	1	1	1	1_j	–

P_ℓ - B-RASP Example: Dyck-1 of Depth 2



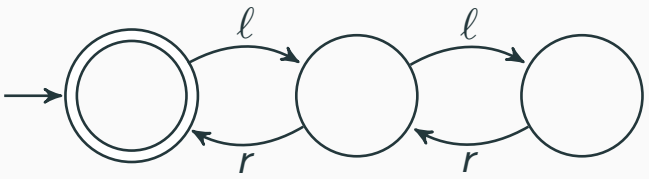
P_ℓ indicates whether the symbol immediately to the left is ℓ.

$$P_\ell(i) = \blacktriangleright_j\,[j < i, 1]\ \ Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_{EOS}	0	0	0	0	0	0
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	0	1	1	0

S_r - B-RASP Example: Dyck-1 of Depth 2



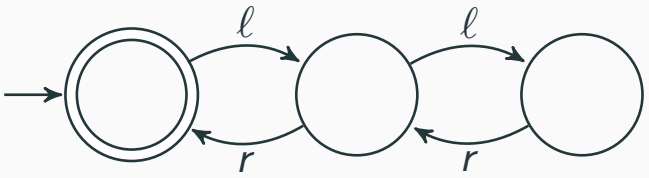
S_r indicates whether the symbol immediately to the right is r.

$$S_r(i) = \blacktriangleleft_j [j > i, 1] \; Q_r(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	0
S_r	0	1	0	1	1	0

I - B-RASP Example: Dyck-1 of Depth 2



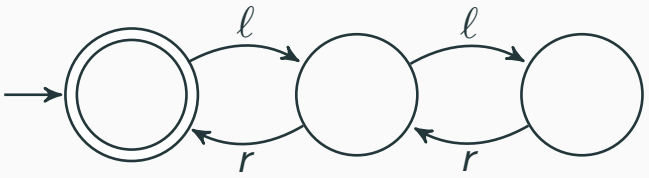
I indicates whether position i is in a consecutive pair ℓr.

$$I(i) = (\, Q_\ell(i) \wedge S_r(i) \,) \vee (\, Q_r(i) \wedge P_\ell(i) \,).$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	0
S_r	0	1	0	1	1	0
I	0	1	1	1	1	0

B_ℓ - B-RASP Example: Dyck-1 of Depth 2



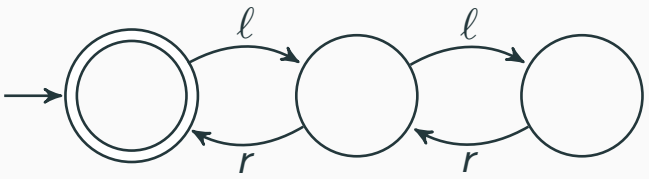
B_ℓ checks that the previous symbol that is not immediately matched is ℓ

$$B_\ell(i) = \blacktriangleright_j \left[j < i, \neg\, I(j) \right] \quad Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	0
S_r	0	1	0	1	1	0
I	0	1	1	1	1	0
B_ℓ	0	1	1	1	1	1

A_r - B-RASP Example: Dyck-1 of Depth 2



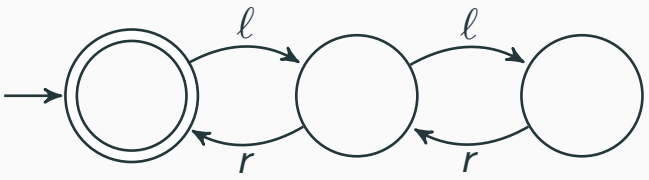
A_r checks that the following symbol that is not immediately matched is r

$$A_r(i) = \blacktriangleright_j \left[j < i, \neg\, I(j) \right] \quad Q_\ell(j) : 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	0
S_r	0	1	0	1	1	0
I	0	1	1	1	1	0
B_ℓ	0	1	0	1	1	0
A_r	1	1	1	1	1	0

C - B-RASP Example: Dyck-1 of Depth 2



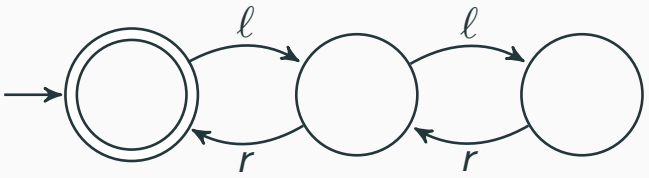
C checks that if the current position is either immediately matched, or is matched as soon as possible

$$C(i) = I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)).$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	0
S_r	0	1	0	1	1	0
I	0	1	1	1	1	0
B_ℓ	0	1	0	1	1	0
A_r	1	1	1	1	1	0
C	1	1	1	1	1	1

Y - B-RASP Example: Dyck-1 of Depth 2



The output vector Y checks that all positions are matched

$$Y(i) = \blacktriangleright_j \left[1, \neg C(j)\right] \; 0:1$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	1	1	0	1	0	0
Q_r	0	0	1	0	1	1
P_ℓ	0	1	1	0	1	0
S_r	0	1	0	1	1	0
I	0	1	1	1	1	0
B_ℓ	0	1	0	1	1	0
A_r	1	1	1	1	1	0
C	1	1	1	1	1	1
Y	1	1	1	1	1	1

$\ell\ell r\ell rr$: Accepted

LTL

A Review of LTL

For input string $\mathbf{w} = \mathbf{w}_1 \cdots \mathbf{w}_n$ and position $i \in [n]$, we define $\mathbf{w}, i \models \phi$ as follows:

$\mathbf{w}, i \models Q_a$	$\mathbf{w}_i = a$
$\mathbf{w}, i \models \phi_1 \vee \phi_2$	$\mathbf{w}, i \models \phi_1$ or $\mathbf{w}, i \models \phi_2$
$\mathbf{w}, i \models \neg\phi_1$	$\mathbf{w}, i \not\models \phi_1$
$\mathbf{w}, i \models \phi_1$ **since** ϕ_2	for some $j < i$, we have $\mathbf{w}, j \models \phi_2$, and for all k such that $j < k < i$, we have $\mathbf{w}, k \models \phi_1$

For an input string $\mathbf{w} \in \Sigma^+$ of length n we write $\mathbf{w} \models \phi$ if and only $\mathbf{w}, n \models \phi$.

A Review of LTL

"I am an a"

	a	b	c	a	b	b	b
Q_a	1	0	0	1	0	0	0
Q_b	0	1	0	0	1	1	1
Q_c	0	0	1	0	0	0	0
$Q_a \vee \neg Q_b$	1	0	1	1	0	0	0
Q_b **since** Q_a	0	1	1	0	1	1	1

$\mathbf{w}, 0 \models Q_a$

"I am an a *or I am not a* b"

	1	b	c	a	b	b	b
Q_a	1	0	0	1	0	0	0
Q_b	0	1	0	0	1	1	1
Q_c	0	0	1	0	0	0	0
$Q_a \vee \neg Q_b$	1	0	1	1	0	0	0
Q_b **since** Q_a	0	1	1	0	1	1	1

$$\mathbf{w}, 3 \vDash Q_a \vee \neg Q_b$$

"I have a left neighbor that is an a *and there are only* b*s between us"*

	a	b	c	a	b	b	b
Q_a	1	0	0	1	0	0	0
Q_b	0	1	0	0	1	1	1
Q_c	0	0	1	0	0	0	0
$Q_a \vee \neg Q_b$	1	0	1	1	0	0	0
Q_b **since** Q_a	0	1	1	0	1	1	1

$\mathbf{w}, 6 \vDash Q_b$ **since** Q_a

Corollaries

Main Idea

This close correspondence between MUHATs and LTL allows us to apply results from one domain to the other.

LTL gives us a way to reason precisely about the computational resources a masked hard-attention transformer uses.

What is the difference in strict vs non-strict masking?

Non-Strictness

Non-strict since is defined as

$$\mathbf{w}, i \models \phi_1 \textbf{ since } \phi_2 \quad \text{for some } j \leq i, \text{ we have } \mathbf{w}, j \models \phi_2, \text{ and for all } k \text{ such that } j < k \leq i, \text{ we have } \mathbf{w}, k \models \phi_1$$

Non-strict masking is the usual masking scheme where a position can attend to itself.

Stutter-Invariant Star-Free Languages

A language over Σ is called *stutter-invariant* iff for all $u, v \in \Sigma^*$ and $a \in \Sigma$, $uav \in L$ iff $uaav \in L$.

An example of a language that is star-free but not stutter-invariant is $\{a\}$.

Peled and Wilke [1997] prove non-strict LTL recognizes exactly the stutter-invariant star-free languages.

How do positional embeddings affect expressive power?

Positional Embeddings

Sinusoidal position embeddings For any even d, let us define a *rational sinusoidal positional embedding* with d dimensions to be a position embedding

$$\mathrm{PE}_n(i) = \begin{bmatrix} \sin 2\pi f_1 i \\ \cos 2\pi f_1 i \\ \cdots \\ \sin 2\pi f_{d/2} i \\ \cos 2\pi f_{d/2} i \end{bmatrix} \qquad f_1, \ldots, f_{d/2} \in \mathbb{Q}.$$

Admittedly, in the original definition, the f_k were not rational.

Modular Predicates

Modular Predicates For any $m \in \mathbb{N}$ and $r \in \{0, \ldots, m-1\}$, we define the *modular predicate* MOD_m^r as follows:

$$w \vDash \mathrm{MOD}_m^r(i) \text{ iff } i \mod m = r$$

Transformers and Regular Languages in AC^0

Say FO[$<, \mathrm{MOD}$] and LTL[MOD] when we add MOD predicates to the logic.

Masked hard-attention transformers are equivalent to FO[$<, \mathrm{MOD}$], which is equivalent LTL[MOD], which is equivalent to the regular languages in AC^0.

Depth

How does adding more layers affect expressive power?

Depth

The *depth* of a masked hard-attention transformer is the number of attention layers it has. The *temporal depth* of an LTL formula is defined inductively as follows:

$$\begin{aligned}
\mathsf{dp}(Q_a) &= 0 \\
\mathsf{dp}(\neg\phi) &= \mathsf{dp}(\phi) \\
\mathsf{dp}(\phi \wedge \psi) &= \max(\mathsf{dp}(\phi), \mathsf{dp}(\psi)) \\
\mathsf{dp}(\phi \ \mathbf{since}\ \psi) &= \max(\mathsf{dp}(\phi), \mathsf{dp}(\psi)) + 1
\end{aligned}$$

Stairs

STAIR_k is the language over $\Sigma = \{a, b, c\}$ of strings which, after deleting c's, contain a^k as a substring.

For instance, $\top$ **since** $(Q_a \wedge (\neg Q_b$ **since** $(Q_a \wedge (\neg Q_b$ **since** $Q_a))))$ recognizes STAIR_3.

- **aa** $\notin \mathrm{STAIR}_3$
- **aaa** $\in \mathrm{STAIR}_3$
- **acaca** $\in \mathrm{STAIR}_3$
- **aaaa** $\in \mathrm{STAIR}_3$
- **acabaca** $\notin \mathrm{STAIR}_3$

The since Hierarchy

Let LTL_k be the set of languages recognizable by LTL formulas of depth at most k. Then, Etessami and Wilke [2000] show:

$$\mathsf{STAIR}_k \in \mathsf{LTL}_k$$
$$\mathsf{STAIR}_{k+1} \notin \mathsf{LTL}_k$$

Depth Hierarchy

$$\mathbf{MUHAT}(\blacktriangleright F)_k \subsetneq \mathbf{MUHAT}(\blacktriangleright F)_{2k+1}$$

Depth Hierarchy

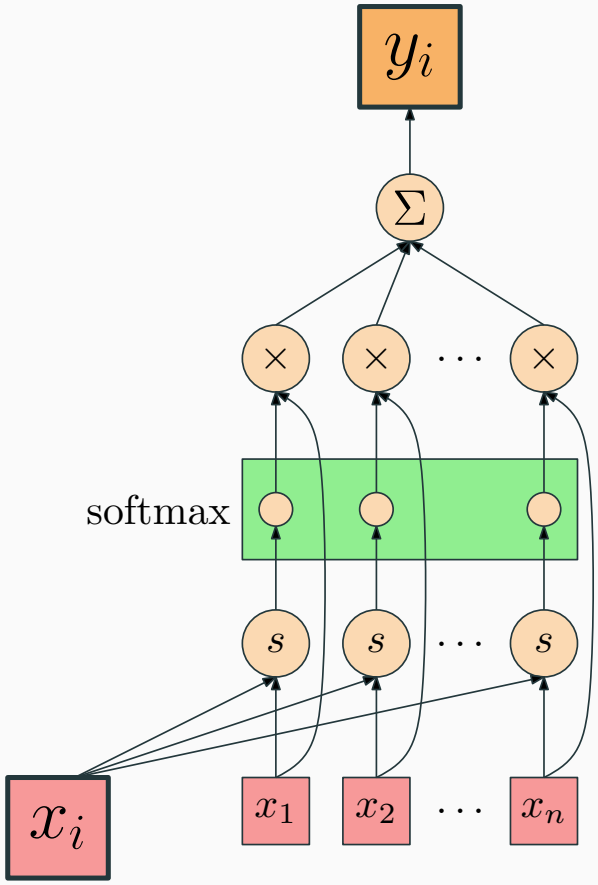
$$\mathbf{MUHAT}(\blacktriangleright F)_k \subseteq \mathsf{LTL}_{2k} \subsetneq \mathsf{LTL}_{2k+1} \subseteq \mathbf{MUHAT}(\blacktriangleright F)_{2k+1}$$

C-RASP

Back at Softmax

Scaled dot-product self-attention with d input/output dimensions and d_{hid} key/value dimensions is a function

$$
\begin{aligned}
att\colon (\mathbb{R}^d)^* &\xrightarrow{\text{lp}} (\mathbb{R}^d)^* \\
\mathbf{X} &\mapsto \mathbf{Y} \\
\mathbf{Q}[i] &= \mathbf{W}^{\mathsf{Q}}(\mathbf{X}[i]) \\
\mathbf{K}[j] &= \mathbf{W}^{\mathsf{K}}(\mathbf{X}[j]) \\
\mathbf{V}[j] &= \mathbf{W}^{\mathsf{V}}(\mathbf{X}[j]) \\
\mathbf{s}[i,j] &= \frac{\mathbf{Q}[i] \cdot \mathbf{K}[j]}{\sqrt{d_{\text{hid}}}} \\
\alpha[i,:] &= \text{softmax}(\mathbf{s}[i,:]) \\
\mathbf{Y}[i] &= \sum_{j \in [n]} \alpha[i,j]\, \mathbf{V}[j]
\end{aligned}
$$



C-RASP: Example Program for Dyck-1

$$
\begin{array}{rcl}
Q_\ell & & \\
Q_r & & \\
C_\ell(i) & := & \#\,[j \le i]\;\; Q_\ell(j) \\
C_r(i) & := & \#\,[j \le i]\;\; Q_r(j) \\
V(i) & := & C_\ell(i) < C_r(i) \\
C_V(i) & := & \#\,[j \le i]\;\; V(j) \\
M(i) & := & C_V(i) = 0 \\
B(i) & := & C_\ell(i) = C_r \\
D(i) & := & M(i) \wedge B(i)
\end{array}
$$

C-RASP Program Trace

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ						
Q_r						
C_ℓ						
C_r						
V						
C_V						
M						
B						
D						□

Initial Vectors - C-RASP Example: Dyck-1

The initial vectors are Q_ℓ and Q_r. These are all defined such that:

$Q_\ell(i)$ = True iff ℓ is i-th symbol

$Q_r(i)$ = True iff r is i-th symbol

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	T	T	F	T	F	F
Q_r	F	F	T	F	T	T

C_ℓ - C-RASP Example: Dyck-1

C_ℓ counts the number of ℓ seen up until and including current position

$$C_\ell(i) = \#[j \leq i] \; Q_\ell(i) \,.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	T	T	F	T	F	F
Q_r	F	F	T	F	T	T
C_ℓ	1	2	2	3	3	3

C_r - C-RASP Example: Dyck-1

C_r counts the number of r seen up until and including current position

$$C_r(i) = \#[j \leq i] \; Q_r(i) \,.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	T	T	F	T	F	F
Q_r	F	F	T	F	T	T
C_ℓ	1	2	2	3	3	3
C_r	0	0	1	1	2	3

V - C-RASP Example: Dyck-1

V indicates a matching violation - if there are ever more r than ℓ

$$V(i) = C_\ell(i) < C_r(i).$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	T	T	F	T	F	F
Q_r	F	F	T	F	T	T
C_ℓ	1	2	2	3	3	3
C_r	0	0	1	1	2	3
V	F	F	F	F	F	F

C_V - C-RASP Example: Dyck-1

C_V counts the number of violations seen up until the current position.

$$C_V(i) = \#[j \leq i] \ \mathrm{V(i)} \, .$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	T	T	F	T	F	F
Q_r	F	F	T	F	T	T
C_ℓ	1	2	2	3	3	3
C_r	0	0	1	1	2	3
V	F	F	F	F	F	F
C_V	0	0	0	0	0	0

M - C-RASP Example: Dyck-1

M checks that the parentheses are always matched by verifying if there are zero violations

$$M(i) = C_V(i) = 0.$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	T	T	F	T	F	F
Q_r	F	F	T	F	T	T
C_ℓ	1	2	2	3	3	3
C_r	0	0	1	1	2	3
V	F	F	F	F	F	F
C_V	0	0	0	0	0	0
M	T	T	T	T	T	T

B - C-RASP Example: Dyck-1

B checks that the parentheses are balanced by verifying if there are equal ℓ as r

$$B(i) = C_\ell(i) = C_r(i)$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	T	T	F	T	F	F
Q_r	F	F	T	F	T	T
C_ℓ	1	2	2	3	3	3
C_r	0	0	1	1	2	3
V	F	F	F	F	F	F
C_V	0	0	0	0	0	0
M	T	T	T	T	T	T
B	T	T	T	T	T	T

D - C-RASP Example: Dyck-1

D checks the string is matched and balanced.

$$D(i) = M(i) \wedge B(i)$$

Program Trace

input	ℓ	ℓ	r	ℓ	r	r
Q_ℓ	T	T	F	T	F	F
Q_r	F	F	T	F	T	T
C_ℓ	1	2	2	3	3	3
C_r	0	0	1	1	2	3
V	F	F	F	F	F	F
C_V	0	0	0	0	0	0
M	T	T	T	T	T	T
B	T	T	T	T	T	T
D	T	T	T	T	T	T

More?

$a^* b^*$ *over* $\Sigma = \{a, b\}$

$$
\begin{aligned}
C_a(i) &:= \#[j \le i]\ Q_a(j) && \#\text{ positions with } a\text{'s} \\
C_b(i) &:= \#[j \le i]\ Q_b(j) && \#\text{ positions with } b\text{'s} \\
V(i) &:= Q_a(i) \wedge C_b(i) \ge 1 && \textit{Violation}\text{: an } a \text{ has } b\text{'s preceding it} \\
C_V(i) &:= \#[j \le i]\ V(j) && \#\ \textit{Violations} \\
Y(i) &:= C_V(i) = 0 && \text{Zero } \textit{Violations}
\end{aligned}
$$

MORE?!

$a^n b^n c^n$ *over* $\Sigma = \{a, b, c\}$

$C_a(i) := \#\,[j \leq i]\ Q_a(j)$	# positions with a's
$C_b(i) := \#\,[j \leq i]\ Q_b(j)$	# positions with b's
$C_c(i) := \#\,[j \leq i]\ Q_c(j)$	# positions with c's
$A(i) := C_b(i) + C_c(i) = 0$	No preceding b's or c's
$B(i) := C_c(i) = 0$	No preceding c's
$C_A(i) := \#\,[j \leq i]\ Q_a(j) \wedge A(j)$	# a's with no preceding b's or c's
$C_B(i) := \#\,[j \leq i]\ Q_b(j) \wedge B(j)$	# b's with no preceding c's
$G_a(i) := C_A(i) = C_a(i)$	no a's have preceding b's or c's
$G_b(i) := C_B(i) = C_b(i)$	no b's have preceding c's
$G_{abc}(i) := C_a(i) = C_b(i) = C_c(i)$	same number of a's, b's, c's
$Y(i) := G_a(i) \wedge G_b(i) \wedge G_{abc}(i)$	Correct order & number of symbols

Ok one more

hello over $\Sigma = \{e, h, l, o\}$

$$C_e(i) := \#[j \le i]\ Q_e(j) \qquad \#\text{ positions with } e\text{'s}$$

$$C_h(i) := \#[j \le i]\ Q_h(j) \qquad \#\text{ positions with } h\text{'s}$$

$$C_l(i) := \#[j \le i]\ Q_l(j) \qquad \#\text{ positions with } l\text{'s}$$

$$C_o(i) := \#[j \le i]\ Q_o(j) \qquad \#\text{ positions with } o\text{'s}$$

$$C_\Sigma(i) := \#[j \le i]\ 1 \qquad \#\text{ symbols in string}$$

$$HE(i) := Q_e(i) \wedge C_h(i) = 1 \qquad \text{A subsequence } he \text{ ends at } i$$

$$C_{he}(i) := \#[j \le i]\ HE(j) \qquad \#\text{ ends of subsequence } he$$

$$HEL(i) := Q_l(i) \wedge C_{he}(i) = 1 \qquad \text{A subsequence } hel \text{ ends at } i$$

$$C_{hel}(i) := \#[j \le i]\ HEL(j) \qquad \#\text{ ends of subsequence } hel$$

$$HELLO(i) := Q_o(i) \wedge C_{hel}(i) = 2 \qquad \text{A subsequence } hello \text{ ends at } i$$

$$Y(i) := HELLO(i) \wedge C_\Sigma(i) = 5 \qquad \text{Length 5 and contains } hello$$

Asking the same questions

- Strict vs non-strict Masking? Unsure what difference this makes here.
- Positional Encodings? Works the same way as in previous case.
- Depth? Unsure. . .

Depth Hierarchy for Log Precision Soft Attention

Seems very related to $\mathbf{TC}^0$ depth hierarchy, as

$$\text{log precision transformers} \subseteq \mathbf{TC}^0$$

From Merrill and Sabharwal [2023]. But if

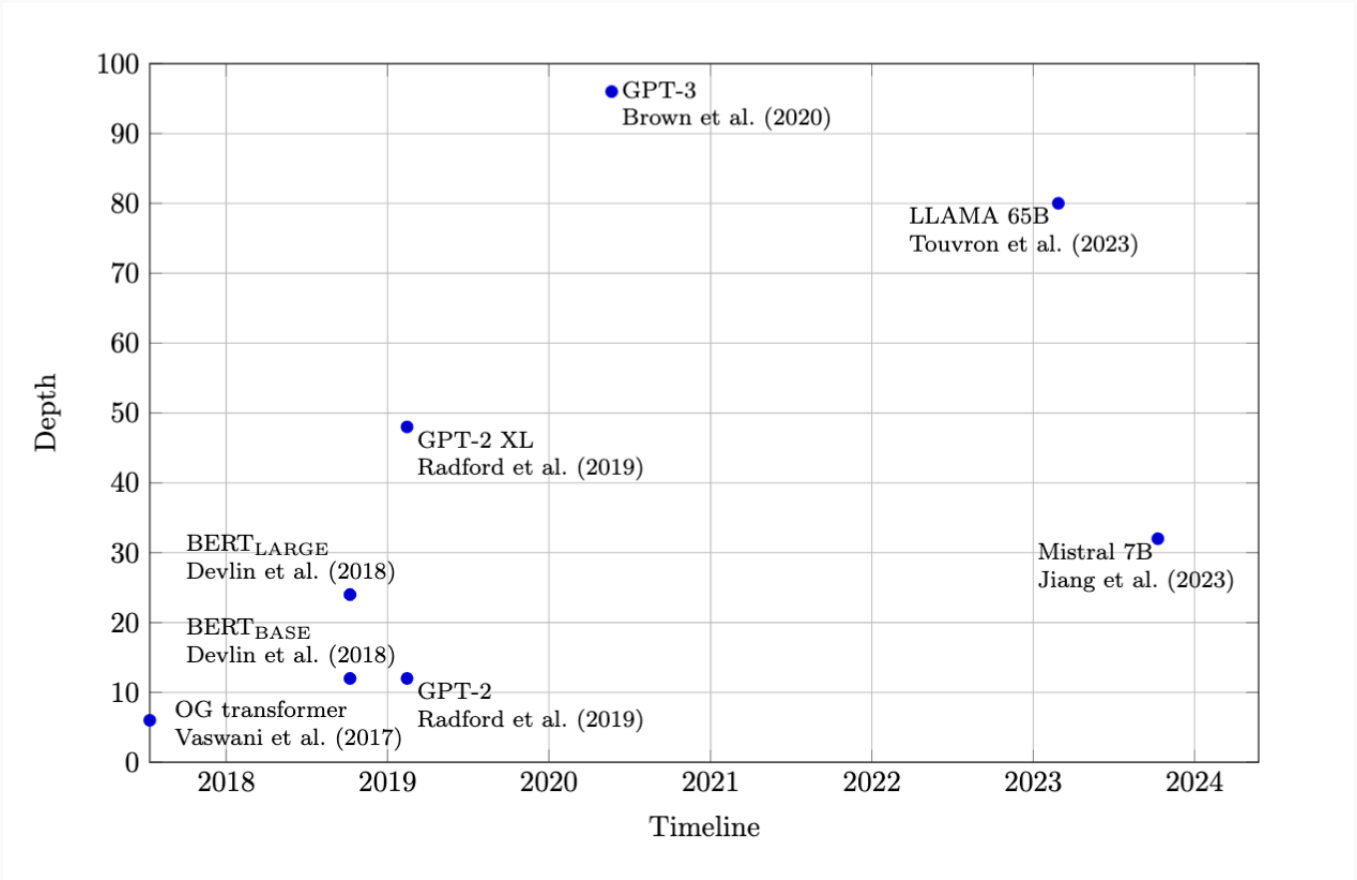
$$\mathbf{TC}^0_d \subsetneq \mathbf{TC}^0_{d+1} \text{ for all } d$$

Then we get

$$\mathbf{TC}^0 \subsetneq \mathbf{NC}^1$$

Which seems to be running into hard questions...

Depth in the Wild



- In feed-forward neural networks with ReLU, two layers are enough
- Transformers have been getting deeper over time

Comparing the Bounds

Incomparability

$$\textbf{B-RASP} \not\subset \textbf{C-RASP}$$

$$\textbf{C-RASP} \not\subset \textbf{B-RASP}$$

Since it seems that without extra positional information:

$$\text{Dyck-1} \in \textbf{C-RASP} \setminus \textbf{B-RASP}$$

$$\Sigma^* aa \Sigma^* \in \textbf{B-RASP} \setminus \textbf{C-RASP}$$

Problems

How to simulate unique hard attention using softmax attention?

- Change the softmax temperature
- Add positional embeddings

We're working on it finding exactly what temperature and what positional embeddings are needed

What Good is Hard Attention?

- Theoretical analysis? But does it predict reality?
- Is it trainable?
- Is it expressible in soft attention?

Today's Goals

1. **Explain** the difference between masked hard-attention transformers and soft attention transformers
2. **Describe** how strict-masking, positional embeddings, and depth contribute to the expressivity of a masked hard-attention transformer
3. **Demonstrate** how the same idea can be applied to softmax transformers
4. **Discuss** the issues with these bounds on expressivity

References i

Satwik Bhattamishra, Kabir Ahuja, and Navin Goyal. On the Ability and Limitations of Transformers to Recognize Formal Languages. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7096–7116, 2020. doi:10.18653/v1/2020.emnlp-main.576.

David Chiang and Peter Cholak. Overcoming a theoretical limitation of self-attention. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7654–7664, May 2022. doi:10.18653/v1/2022.acl-long.527. URL https://aclanthology.org/2022.acl-long.527.

Javid Ebrahimi, Dhruv Gelda, and Wei Zhang. How can self-attention networks recognize Dyck-n languages? In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4301–4306, November 2020. doi:10.18653/v1/2020.findings-emnlp.384.

Kousha Etessami and Thomas Wilke. An until hierarchy and other applications of an ehrenfeucht–fraıssé game for temporal logic. *Information and Computation*, 160(1-2):88–108, 2000. doi:10.1006/inco.1999.2846.

Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient transformers via top-k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pages 39–52, 2021. doi:10.18653/v1/2021.sustainlp-1.5.

Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020. doi:10.1162/tacl_a_00306. URL https://aclanthology.org/2020.tacl-1.11.

Jambay Kinley. *Two-Stream Transformer Architecture With Discrete Attention for Better Interpretrability and Separation of Model Concerns*. Bachelor's thesis, Harvard College, 2020. URL https://dash.harvard.edu/handle/1/37364732.

References ii

William Merrill and Ashish Sabharwal. A logic for expressing log-precision transformers. In *Advances in Neural Information Processing Systems*, volume 36, pages 52453–52463, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a48e5877c7bf86a513950ab23b360498-Abstract-Conference.html.

William Merrill, Vivek Ramanujan, Yoav Goldberg, Roy Schwartz, and Noah A. Smith. Effects of parameter norm growth during transformer training: Inductive bias from gradient descent. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1766–1781, 2021. doi:10.18653/v1/2021.emnlp-main.133. URL https://aclanthology.org/2021.emnlp-main.133.

Doron Peled and Thomas Wilke. Stutter-invariant temporal properties are expressible without the next-time operator. *Information Processing Letters*, 63(5):243–246, 1997. doi:10.1016/S0020-0190(97)00133-6.

Hongfei Xu, Qiuhui Liu, Josef van Genabith, and Deyi Xiong. Learning hard retrieval decoder attention for Transformers. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 779–785, 2021. doi:10.18653/v1/2021.findings-emnlp.67.